

HURDLE 2

Data Storage on Azure

Upload your dataset to Azure Blob Storage and query it from Python.

 30 Minutes

Objective

Move your enriched CSV dataset off your laptop and into Azure cloud storage. Verify that your Python application can read the data back from the cloud — simulating how a real production pipeline would retrieve data at runtime.

Background

In production, raw and processed data are never stored on application servers. Azure Blob Storage provides cheap, durable, and scalable object storage — the natural home for CSV files, model artefacts, and exports. Every major GreenOps workflow (ingestion, forecasting, reporting) will read from this store.

Tasks

Task A — Create Azure Storage Account

1. In the Azure Portal, go to Storage Accounts → Create.
2. Select your greenops-rg-[roll-number] resource group.
3. Name: greenopsstorage[rollnumber] (lowercase, no hyphens, globally unique).
4. Performance: Standard | Redundancy: LRS (Locally Redundant — lowest cost for dev).
5. Click Review + Create → Create.

Task B — Create a Container and Upload

6. Inside the Storage Account, go to Containers → + Container.
7. Name: greenops-data | Access level: Private.
8. Upload cloud_usage_enriched.csv from your local data/ folder into this container.
9. Take a screenshot of the file visible in the Azure portal.

Task C — Read from Blob via Python

10. In the Azure Portal → Storage Account → Access Keys → copy Connection String.
11. Create `data/azure_reader.py` with the following pattern:

```
from azure.storage.blob import BlobServiceClient import pandas as pd, io
CONN_STR = "<paste-your-connection-string>" client =
BlobServiceClient.from_connection_string(CONN_STR) blob =
client.get_container_client("greenops-
data") .get_blob_client("cloud_usage_enriched.csv") data =
blob.download_blob().readall() df = pd.read_csv(io.BytesIO(data))
print(df.shape) print(df['co2e_kg'].sum())
```

12. Run the script. Confirm the shape and CO2e total match Hurdle 1 output.
13. IMPORTANT: Never commit your connection string. Add a `.env` file and use `python-dotenv`, or use environment variables.

Task D — (Stretch) Azure Cosmos DB or SQL

If time allows: create a free-tier Azure Cosmos DB (Core SQL API) account, create a `greenops` database with a `carbon_records` container, and insert the top 10 rows from the DataFrame using the `azure-cosmos` SDK.



Concept Check

- What is the difference between Azure Blob Storage and Azure SQL Database? When would you choose each?
- What is LRS replication and what are its limitations vs GRS?
- Why is it a security risk to hardcode a connection string in source code?



Self-Evaluation Checklist

☐	Azure Storage Account created in correct resource group and region
☐	<code>greenops-data</code> container created; CSV uploaded successfully
☐	<code>azure_reader.py</code> reads the blob and prints correct shape and CO2e total
☐	Connection string is NOT committed to GitHub (using <code>.env</code> or env var)
☐	Screenshot of file in Azure portal saved
☐	Concept check answered

➡ Proceed to Hurdle 3 once checklist is complete.